Module 6 - Project Retrospective

Group 19 | Real-Time Network Intrusion Detection Using Machine Learning

MODULE 6 Assignment | AI Deployment, Monitoring, Support & Maintenance

Achievements

We completed a five-class NSL-KDD classification workflow and assembled it into a reusable prototype package. The implementation includes label grouping, a stratified validation split, preprocessing within a pipeline, SMOTENC for training-fold imbalance handling, Decision Tree and Random Forest baselines, an XGBoost comparison, three-fold GridSearchCV, an exported-pipeline workflow, and evaluation on the separate KDDTest+ benchmark. We also prepared a Streamlit CSV interface, an optional FastAPI endpoint, input-validation tests, run instructions and a proposed deployment/maintenance plan.

Challenges and lessons learned

The most important challenge was the large gap between validation and independent-test performance. The selected model achieved 99.89% validation accuracy and 0.9825 Macro-F1, but only 74.63% accuracy and 0.5155 Macro-F1 on KDDTest+. The 0.06 R2L recall and 0.10 U2R recall show that excellent-looking validation metrics can hide weak generalization on rare or shifted attack categories. We learned to preserve a genuinely independent test set, report macro and per-class measures, keep resampling inside the training pipeline, and distinguish benchmark results from operational evidence.

Limitations

The current classifier is not ready to make security decisions on live traffic. It uses an older public benchmark, has poor recall for two attack groups, and has a high false-positive rate for the Normal class on KDDTest+. It does not capture packets, block traffic, provide continuous monitoring or operate as a production security service. The model binary was not included in the files supplied for this packaging pass, so end-to-end inference with the trained weights and public app hosting still require a verified model export and deployment test.

Planned improvements

Our next steps are to regenerate and checksum the trained artifact, confirm that inference results match the notebook, deploy and test the Streamlit interface for public access, and investigate R2L/U2R errors using training-only cross-validation. We will compare cost-sensitive models and calibrated outputs, use more representative contemporary traffic data where authorized, and assess drift with verified labels. Any future pilot will require analyst oversight, documented rollback, privacy safeguards and independent security review.

Reflection

The project has helped us connect model development with the practical work required to package, test, document and maintain an AI application. The main lesson is that a working interface is not enough: the model must be reproducible, its test limitations must be transparent, and the deployment plan must distinguish what is implemented from what is proposed.